

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №3
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Динамические списки.»

Выполнили: ст. гр. 25BBB2

Чураева А.А.

Доблер Б.С.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В.

Пенза 2026

Цель работы

Изучить принципы организации динамических структур данных на языке С, освоить работу с односвязными списками и на их основе реализовать три классические структуры:

1. приоритетную очередь;
2. обычную очередь (FIFO);
3. стек (LIFO).

Постановка задачи

В соответствии с заданием лабораторной работы требуется:

1. Реализовать приоритетную очередь путём добавления элемента в список в соответствии с его приоритетом: объект с большим приоритетом должен располагаться перед объектом с меньшим приоритетом.
2. На основе приведённого в методических указаниях кода реализовать структуру данных «Очередь».
3. На основе того же кода реализовать структуру данных «Стек».

Все три структуры поддерживают базовые операции:

- добавление элемента;
- извлечение элемента;
- просмотр содержимого;
- освобождение всей занятой памяти при завершении работы программы.

Теоретические сведения

Динамические структуры данных

Динамические структуры отличаются от статических двумя ключевыми свойствами:

1. Каждый элемент должен содержать информацию, логически связывающую его с другими элементами структуры (в односвязном списке — указатель на следующий элемент).

2. Размер структуры заранее неизвестен и определяется во время выполнения программы. Для управления памятью применяются указатели и функции `malloc / free`.

Односвязный список

Простейшая динамическая структура — односвязный линейный список. Каждый его элемент описывается структурой вида:

```
struct node
{
    char inf[256];
    struct node *next;
};
```

Доступ к списку осуществляется через указатели `head` (первый элемент) и `last` (последний элемент). Для списка реализуются функции создания, добавления, удаления, поиска и просмотра элементов.

Приоритетная очередь — структура, в которой каждый элемент имеет числовой приоритет. Извлечение всегда происходит для элемента с наибольшим (или наименьшим — по договорённости) приоритетом. В данной работе: **чем больше число, тем выше приоритет**. При добавлении новый элемент вставляется не в конец, а в то место списка, где нарушается убывающий порядок приоритетов.

Очередь работает по принципу *First In — First Out*: первым приходит — первым уходит. Добавление производится строго в конец (`last->next = p; last = p;`), а извлечение — строго из начала (`head = head->next; free(temp);`). Благодаря указателю `last` добавление выполняется за $O(1)$.

Стек работает по принципу *Last In — First Out*: последним пришёл — первым ушёл. Добавление (`push`) и извлечение (`pop`) выполняются с одного конца — вершины стека. В реализации используется только один указатель `head` на вершину. Указатель на конец списка (`last`) не нужен.

Псевдокод

В данном разделе приведён псевдокод всех ключевых операций для трёх структур данных.

Общие операции для всех структур

Создание нового элемента (get_struct)

ФУНКЦИЯ get_struct:

 ВЫДЕЛИТЬ память под узел p

 ЕСЛИ p = NULL ТО

 ВЫВЕСТИ "Ошибка выделения памяти"

 ЗАВЕРШИТЬ программу с кодом 1

 КОНЕЦ ЕСЛИ

 ВЫВЕСТИ "Введите название объекта: "

 СЧИТАТЬ строку в p.inf (не более 255 символов)

 ЕСЛИ строка пуста ТО

 p.inf = "" (пустая строка)

 КОНЕЦ ЕСЛИ

 ОЧИСТИТЬ буфер ввода

 p.next = NULL

 ВЕРНУТЬ p

КОНЕЦ ФУНКЦИИ

Просмотр содержимого (review)

ФУНКЦИЯ review:

 ЕСЛИ head = NULL ТО

 ВЫВЕСТИ "Структура пуста"

 ВЕРНУТЬСЯ

 КОНЕЦ ЕСЛИ

 cur = head

ПОКА $cur \neq \text{NULL}$ ЦИКЛ

ВЫВЕСТИ $cur.inf$, $cur.priority$ (если есть)

$cur = cur.next$

КОНЕЦ ЦИКЛА

КОНЕЦ ФУНКЦИИ

Освобождение памяти (clear_list)

ФУНКЦИЯ clear_list:

ПОКА $head \neq \text{NULL}$ ЦИКЛ

$temp = head$

$head = head.next$

ОСВОБОДИТЬ $temp$

КОНЕЦ ЦИКЛА

$last = \text{NULL}$

КОНЕЦ ФУНКЦИИ

Главное меню (общая схема)

ФУНКЦИЯ main:

$choice = -1$

ПОКА $choice \neq 0$ ЦИКЛ

ВЫВЕСТИ меню

СЧИТАТЬ $choice$

ЕСЛИ ввод некорректен ТО

ОЧИСТИТЬ буфер

$choice = -1$

ПРОДОЛЖИТЬ

КОНЕЦ ЕСЛИ

ОЧИСТИТЬ буфер

ВЫБОР $choice$:

1: добавить элемент

2: извлечь элемент

3: просмотр

0: clear_list

ИНАЧЕ: "Неверный ввод"

КОНЕЦ ЦИКЛА

КОНЕЦ ФУНКЦИИ

Приоритетная очередь

Добавление по приоритету (spstore)

ФУНКЦИЯ spstore:

p = get_struct()

ЕСЛИ p = NULL ТО ВЕРНУТЬСЯ

ЕСЛИ head = NULL ТО

head = p

last = p

ВЕРНУТЬСЯ

КОНЕЦ ЕСЛИ

prev = NULL

cur = head

// пропускаем все элементы, у которых приоритет не меньше нашего

ПОКА cur ≠ NULL И cur.priority ≥ p.priority ЦИКЛ

prev = cur

cur = cur.next

КОНЕЦ ЦИКЛА

ЕСЛИ prev = NULL ТО

// приоритет нового выше, чем у головы — вставка в начало

p.next = head

head = p

ИНАЧЕ

// вставка после prev

p.next = prev.next

prev.next = p

ЕСЛИ p.next = NULL TO last = p

КОНЕЦ ЕСЛИ

КОНЕЦ ФУНКЦИИ

Извлечение элемента с максимальным приоритетом (serve)

ФУНКЦИЯ serve:

ЕСЛИ head = NULL TO

ВЫВЕСТИ "Очередь пуста"

ВЕРНУТЬСЯ

КОНЕЦ ЕСЛИ

temp = head

head = head.next

ЕСЛИ head = NULL TO last = NULL

ВЫВЕСТИ "Извлечён объект:", temp.inf, "(Приоритет:", temp.priority, ")"

ОСВОБОДИТЬ temp

КОНЕЦ ФУНКЦИИ

Очередь (FIFO)

Постановка в конец (spstore)

ФУНКЦИЯ spstore:

p = get_struct()

ЕСЛИ head = NULL TO

head = p

last = p

ИНАЧЕ

```
// цепляем к хвосту и обновляем last
```

```
last.next = p
```

```
last = p
```

КОНЕЦ ЕСЛИ

КОНЕЦ ФУНКЦИИ

Извлечение из начала (serve)

ФУНКЦИЯ serve:

```
ЕСЛИ head = NULL ТО
```

```
    ВЫВЕСТИ "Очередь пуста"
```

```
    ВЕРНУТЬСЯ
```

КОНЕЦ ЕСЛИ

```
temp = head
```

```
head = head.next
```

```
ЕСЛИ head = NULL ТО last = NULL
```

```
    ВЫВЕСТИ "Извлечен объект:", temp.inf
```

```
    ОСВОБОДИТЬ temp
```

КОНЕЦ ФУНКЦИИ

Стек (LIFO)

Добавление на вершину (push)

ФУНКЦИЯ push:

```
p = get_struct()
```

```
p.next = head
```

```
head = p
```

КОНЕЦ ФУНКЦИИ

Снятие с вершины (pop)

ФУНКЦИЯ pop:

```
ЕСЛИ head = NULL ТО
```

ВЫВЕСТИ "Стек пуст"

ВЕРНУТЬСЯ

КОНЕЦ ЕСЛИ

temp = head

head = head.next // вершиной становится нижележащий элемент

ВЫВЕСТИ "Извлечен объект:", temp.inf

ОСВОБОДИТЬ temp

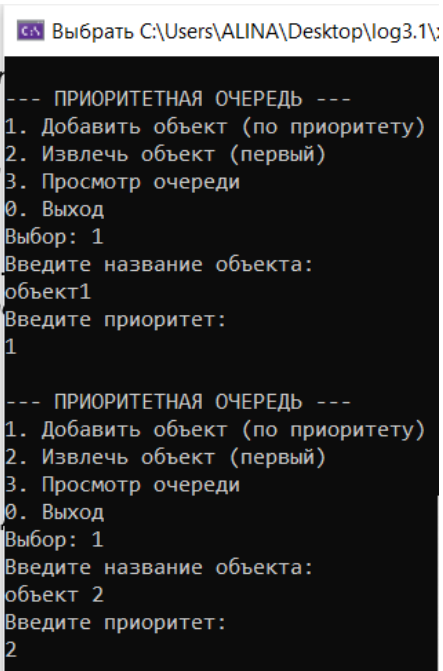
КОНЕЦ ФУНКЦИИ

Результаты работы программы

Ниже приведены скриншоты работы каждой из трёх программ на различных этапах.

Приоритетная очередь

Добавление нескольких элементов с разными приоритетами.



```
Выбрать C:\Users\ALINA\Desktop\log3.1\
--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить объект (по приоритету)
2. Извлечь объект (первый)
3. Просмотр очереди
0. Выход
Выбор: 1
Введите название объекта:
объект1
Введите приоритет:
1
--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить объект (по приоритету)
2. Извлечь объект (первый)
3. Просмотр очереди
0. Выход
Выбор: 1
Введите название объекта:
объект 2
Введите приоритет:
2
```

Просмотр очереди — элементы отсортированы по убыванию приоритета.

```
--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить объект (по приоритету)
2. Извлечь объект (первый)
3. Просмотр очереди
0. Выход
Выбор: 3
Имя - объект, Приоритет - 2
Имя - объект1, Приоритет - 1
```

Извлечение элемента с наибольшим приоритетом.

```
--- ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ---
1. Добавить объект (по приоритету)
2. Извлечь объект (первый)
3. Просмотр очереди
0. Выход
Выбор: 2
Извлечён объект: объект (Приоритет: 2)
```

Очередь FIFO

Постановка объектов в очередь.

```
--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---
1. Добавить объект (в конец)
2. Извлечь объект (из начала)
3. Просмотр очереди
0. Выход
Выбор: 3
Объект: объект 1
Объект: объект 2
```

Просмотр очереди — порядок совпадает с порядком добавления.

```
--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---
1. Добавить объект (в конец)
2. Извлечь объект (из начала)
3. Просмотр очереди
0. Выход
Выбор: 3
Объект: объект 1
Объект: объект 2
```

Извлечение объекта из начала очереди.

```
--- ОБЫЧНАЯ ОЧЕРЕДЬ (FIFO) ---
1. Добавить объект (в конец)
2. Извлечь объект (из начала)
3. Просмотр очереди
0. Выход
Выбор: 2
Извлечен объект: объект 1
```

Стек LIFO

Добавление объектов на вершину стека.

```
C:\Users\ALINA\Desktop\log3.3\x64\Debug\log:
--- СТЕК (LIFO) ---
1. Добавить объект (на вершину)
2. Извлечь объект (с вершины)
3. Просмотр стека
0. Выход
Выбор: 1
Введите название объекта: объект 1

--- СТЕК (LIFO) ---
1. Добавить объект (на вершину)
2. Извлечь объект (с вершины)
3. Просмотр стека
0. Выход
Выбор: 1
Введите название объекта: объект 2
```

Просмотр стека — вершина сверху.

```
--- СТЕК (LIFO) ---
1. Добавить объект (на вершину)
2. Извлечь объект (с вершины)
3. Просмотр стека
0. Выход
Выбор: 3
Объект: объект 2 <-- вершина
Объект: объект 1
```

Снятие объекта с вершины.

```
--- СТЕК (LIFO) ---
1. Добавить объект (на вершину)
2. Извлечь объект (с вершины)
3. Просмотр стека
0. Выход
Выбор: 2
Извлечен объект: объект 2
```

Попытка снятия с пустого стека.

```
--- СТЕК (LIFO) ---  
1. Добавить объект (на вершину)  
2. Извлечь объект (с вершины)  
3. Просмотр стека  
0. Выход  
Выбор: 2  
Стек пуст
```

Вывод:

В результате выполнения лабораторной работы были изучены принципы организации динамических структур данных на языке C, освоена работа с односвязными списками: создание, добавление, извлечение, просмотр, освобождение памяти, реализованы три классические структуры — приоритетная очередь, очередь (FIFO) и стек (LIFO) — на единой базе односвязного списка, показано, что выбор конкретной структуры определяется **дисциплиной доступа** к концам списка, закреплены навыки обработки некорректного пользовательского ввода и безопасной работы с буфером консоли.